Universidad del Valle de Guatemala

Programación Orientada a Objetos

Segundo Ciclo 2026



Laboratorio #2

Erwin Tomás 26234

Sección 20

30/09/2026

¿Qué propiedades y métodos tendrá cada clase? ¿Qué tipo deben tener las propiedades y métodos de cada clase? ¿Qué parámetros serán requeridos por los métodos en sus clases?

Aquí contesté las preguntas 1, 2 y 6

Clase: Ubicación - Tiene los 5 lugares físicos que se están investigando

Propiedades:

codigo: String

nombre: String

direccion: String

nivelRiesgo: int

estado: String

Métodos:

Ubicacion: Constructor

toString (): String

Getters y setters para todos los atributos

Clase: Pista – Tiene todas las evidencias que se recopilan para el caso

Propiedades:

codigo: String

descripcion: String

tipoEvidencia: String

nivelImportancia: int

nivelConfiabilidad: int

Métodos:

Pista: Constructor

toString (): String

Getters y setters para los atributos

Clase: Caso – Es como el gestor que tiene toda la información

Propiedades:

nombreCaso: String

codigoIdentificacion: String

detectiveResponsable: String

ubicaciones: Ubicacion[] (arreglo)

pistas: ArrayList <Pista>

Métodos:

Caso: constructor

registrarUbicacion (int posicion, Ubicacion ubicacion): boolean

obtenerUbicacion (int posicion): Ubicacion

modificarUbicacion (int posicion, int nuevoNivelRiesgo, String nuevoEstado): boolean

descartarUbicacion (int posicion): boolean

obtenerTodasUbicaciones (): ArrayList<Ubicacion>

registrarPista (Pista pista): boolean

buscarPistaPorCodigo (String codigo): Pista

modificarPista (String codigo, String NuevaDescripcion, String nuevoTipoEvidencia, int nuevaImportancia, int nuevaConfiabilidad): boolean

eliminarPista (String codigo): boolean

obtenerTodasPistas (): ArrayList<Ubicacion>

cantidadUbicacionesRegistradas(): int

espaciosDisponiblesUbicaciones(): int

ubicacionMayorRiesgo(): Ubicación

cantidadPistasRegistradas(): int

pistaMayorImportancia(): int

pistaMayorConfiabilidad(): Pista

promedioImportanciaPistas(): double

Clase: Main

Métodos:

iniciarPrograma(): void

solicitarDatosInicialesCaso(): void

ejecutarMenu(): void

procesarRegistroUbicacion(): void

mostrarUbicacionEspecífica(): void

mostrarTodasUbicaciones(): void

procesarModificacionUbicacion(): void

procesarDescarteUbicacion(): void

procesarRegistroPista(): void

mostrarTodasPistas(): void

procesarBusquedaPista(): void

procesarModificacionPista(): void

procesarEliminacionPista(): void

mostrarEstadisticasYReportes(): void

leerEnteroValidado(String mensaje): int

leerCadenaNoVacia(String mensaje): String

¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?

ubicaciones, que está en la clase de Caso.

Tendrá objetos de tipo Ubicacion, lo que contiene nombre, dirección, nivel de riesgo y estado, que serán las 5 ubicaciones que se podrán registrar, por esto su tamaño será fijo para 5 elementos.

¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?

Pistas, que está en la clase de Caso. Tendrá elementos de la clase de Pista, toda la información de cada pista, como nivel de confiabilidad y código, entre otra información.

¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?

Para todos los atributos sin importar la clase serán private.

Casi todos los métodos incluyendo constructores serán public.

Solo los métodos de validación serán private.

¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?

Los valores iniciales se podrán asignar usando los constructores de cada objeto. El arreglo de Ubicaciones tendrá 5 espacios con valor null y el arraylist de Pista estará vacío. Entre las validaciones están:
El nivel de riesgo de una ubicación, que tiene que estar dentro del rango de 1 a 10.

El nivel de importancia de una pista, que tiene que estar dentro de 1 a 10.

El nivel de confiabilidad de una pista, dentro de 1 a 100.

El código de las pistas, que tiene que ser único y por lo que no puede existir previamente.

Que la posición en el arreglo sea válida, que esté dentro del rango y que no esté ocupada, en caso de querer agregar algo o que no esté vacía en caso de querer modificarlo o mostrarlo.

¿Cómo determinará si una posición del arreglo contiene una ubicación o contiene null?

Para verificar si una posición está vacía usaré una condición que diga que si esa ubicación es == a null entonces ese espacio está vacío y si no (else) que indique que sí contiene algo.

¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?

La búsqueda la haré con un ciclo for que recorra el arreglo hasta encontrar una pista que tenga el mismo código que el que se está buscando y después un return que regrese esa pista, si no la encuentra que regrese null. Para modificar una pista primero se busca la pista con la búsqueda y luego se usa set() . Para eliminar se busca la pista y cuando se encuentra se elimina usando remove().

¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.

Al intentar agregar un valor que esté fuera de rango, como para nivel de riesgo, nivel de importancia y si se intenta registrar un índice fuera del rango, se producirá una excepción del tipo IllegalArgumentException.

Si el usuario intenta ingresar texto en una opción del menú se producirá una excepción del tipo InputMismatchException.

Las otras como IndexOutofBounds y NullPointer ya no pasarán por haber agregado validaciones de los valores.

Los bloques de try-catch-finally irán en el Main, cuando se pida al usuario ingresar valores y en el registro y modificación de objetos.

Diagrama UML

